효율적인 프로젝트 협업을 위한

Git & GitHub 기초와 프로젝트 환경 구성

Git & GitHub
버전 관리 · 협업 · 프로젝트 문서화

목차

01. 버전 관리와 Git의 이해
    1. 버전 관리의 이해
    2. Git과 GitHub의 이해
    3. Git의 구조와 작업 흐름

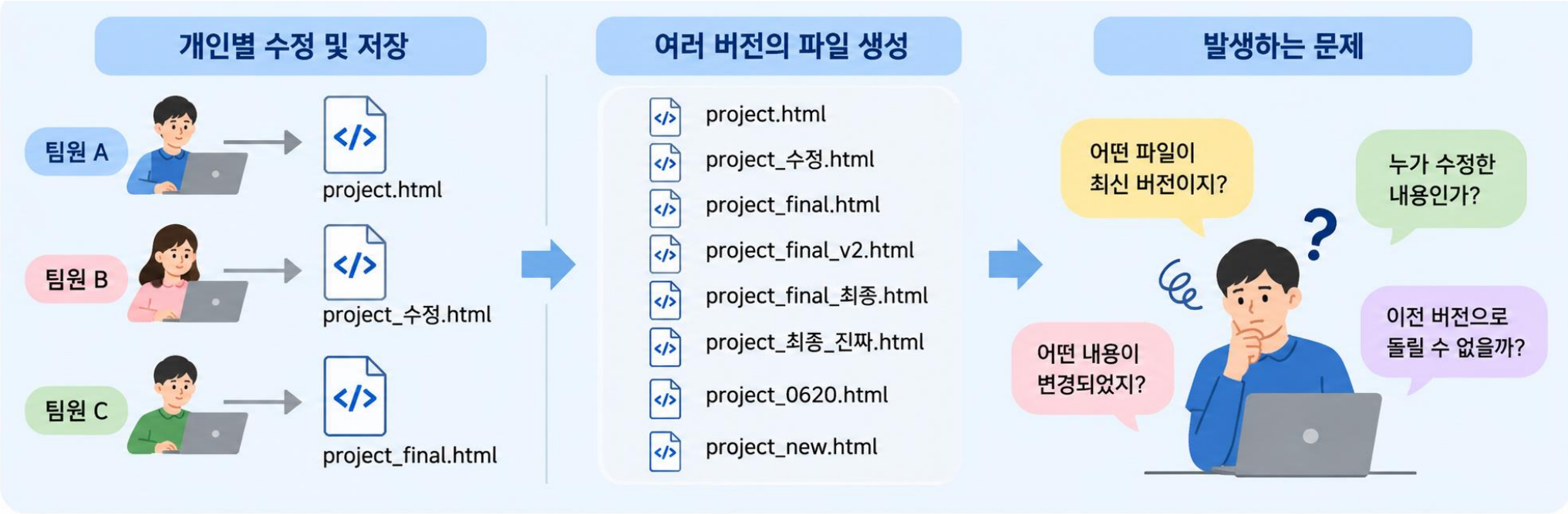
02. 개발 환경과 Git 활용
    4. GitHub 저장소와 개발 환경
    5. Git 기본 명령어
    6. Branch와 Merge를 활용한 협업

03. 프로젝트 문서화와 GitHub 실습
    7. README.md와 .gitignore 파일 설정
    8. GitHub 프로젝트 저장소 구성 실습

01 버전 관리와 Git의 이해

■ 프로젝트 파일 관리에서 발생하는 문제

- 파일 중복: 수정할 때마다 새로운 파일이 생성되어 여러 버전이 혼재
- 변경 이력 확인의 어려움: 수정 시점과 변경 내용을 추적하고 비교하기 어려움
- 협업 충돌: 여러 팀원이 동일한 파일을 수정하여 작업 결과 통합에 어려움 발생
- 복구의 어려움: 잘못된 수정이나 덮어쓰기 발생 시 이전 상태로 복원하기 어려움



01 버전 관리와 Git의 이해

■ 버전 관리(Version Control)의 개념

- 파일이나 프로젝트의 변경 사항을 기록하고 체계적으로 관리하는 방법
- 수정 시점, 변경 내용, 작업자 등 변경 이력을 관리
- 이전 버전과 현재 버전의 차이를 비교하고 필요시 복구
- 여러 개발자의 작업 내용을 공유하고 통합하여 협업 지원
- 버전 관리 시스템(Version Control System, VCS)의 유형

구분	관리 방식	주요 활용 환경	대표 도구
로컬 방식 (Local VCS)	개인 PC에서 변경 이력 관리	개인 파일 및 버전 관리	RCS (Revision Control System)
중앙집중식 (CVCS)	중앙 서버에서 변경 이력 통합 관리	기업 내부 시스템, 레거시 프로젝트	Subversion (Apache Subversion)
분산 방식 (DVCS)	개발자별 저장소에서 변경 이력 관리	신규 소프트웨어 개발, 오픈소스 및 협업 프로젝트	Git

01 버전 관리와 Git의 이해

Git의 개념

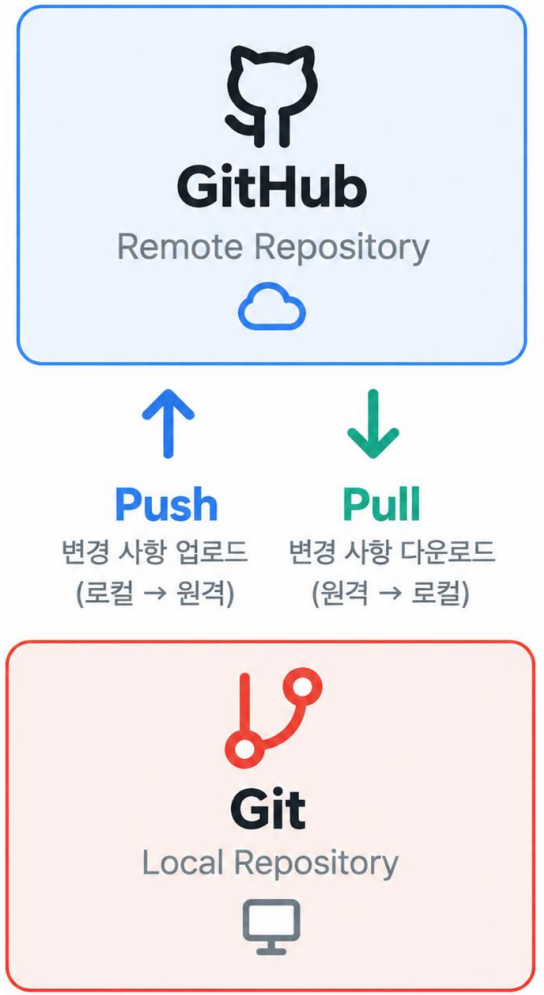
- 소스코드의 변경 이력을 기록하고 관리하는 분산 버전 관리 시스템(DVCS)
- 개발자별 로컬 저장소에서 버전 관리 및 작업 수행
- Commit을 통해 변경 사항을 기록하고 이전 버전 확인 및 복구
- Branch와 Merge를 활용한 독립적인 개발 및 변경 사항 통합

https://git-scm.com/

GitHub의 개념

- Git 저장소를 온라인에서 관리하고 공유하는 협업 플랫폼
- 원격 저장소(Remote Repository)를 통한 소스코드 공유 및 공동 개발
- Pull Request, Issues 등 프로젝트 협업 및 관리 기능 제공

https://github.com/

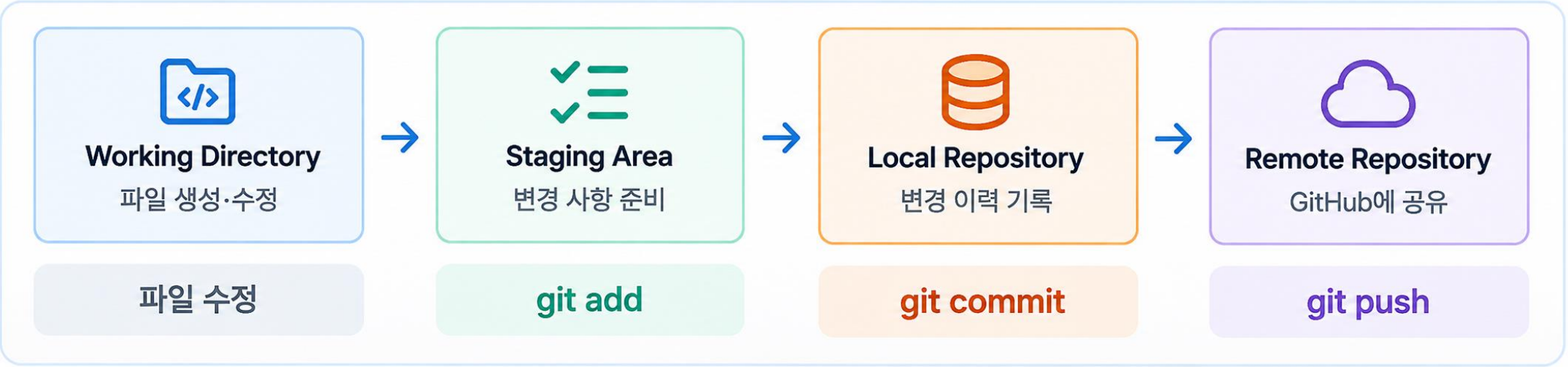


01 버전 관리와 Git의 이해

■ Git의 주요 작업 영역

- 작업 디렉터리(Working Directory): 개발자가 파일을 생성하고 수정하는 작업 공간
- 스테이징 영역(Staging Area): Commit할 변경 사항을 선택하고 준비하는 공간
- 로컬 저장소(Local Repository): Commit을 통해 변경 이력을 기록하고 관리하는 저장소
- 원격 저장소(Remote Repository): GitHub 등을 통해 변경 이력을 공유하고 협업하는 저장소

■ Git의 기본 작업 흐름



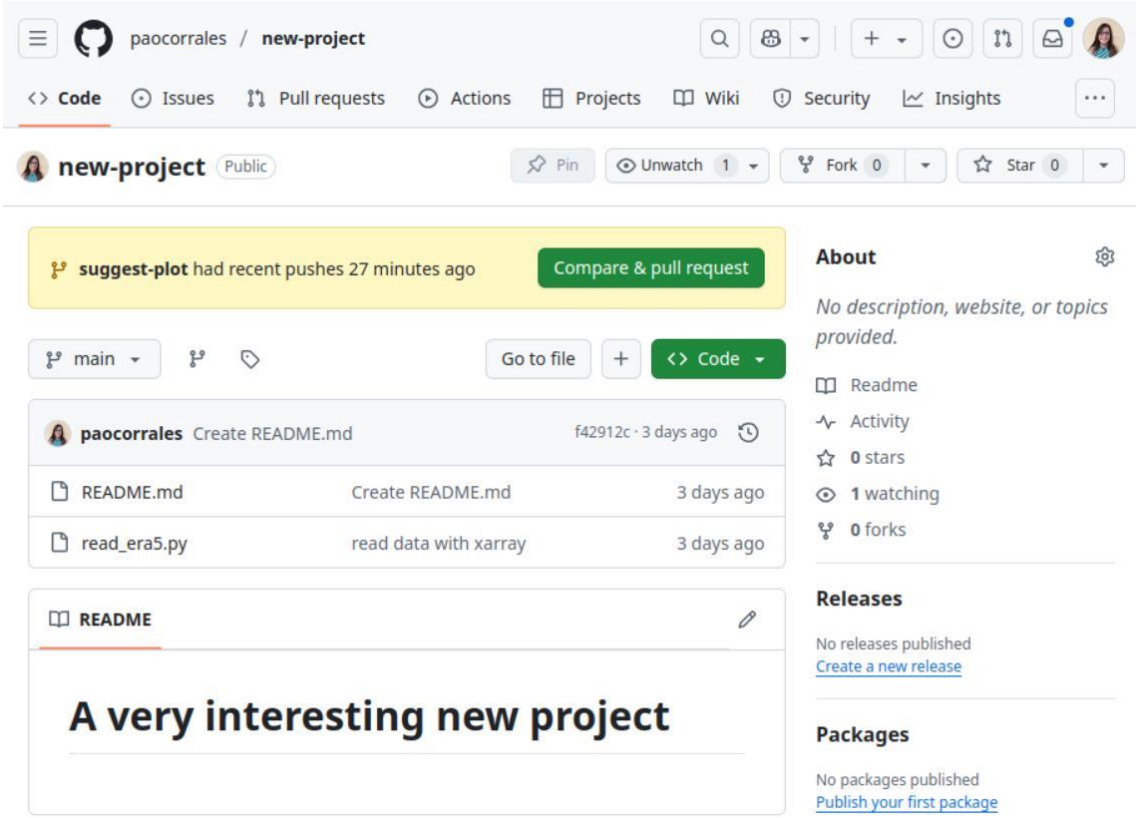
02 개발 환경과 Git 활용

GitHub 저장소(Repository)의 이해

- 프로젝트의 소스코드와 변경 이력을 온라인에서 관리하는 공간
- 프로젝트별로 독립적인 저장소(Repository) 생성 및 관리
- 저장소 URL을 통해 프로젝트 공유 및 협업 지원

GitHub 저장소의 주요 구성

- Code: 프로젝트 파일 및 폴더 관리
- Commits: 소스코드 변경 이력 확인
- Branches: 기능별 독립적인 개발 작업 관리
- Issues: 오류, 개선 사항 및 작업 항목 관리
- Pull Requests: 변경 사항 검토 및 병합 요청



GitHub 저장소 화면 예시 - https://github.com/

02 개발 환경과 Git 활용

■ GitHub 저장소 생성 및 공개 범위 설정

- GitHub 저장소(Repository) 생성
  - GitHub 로그인 후 프로필 클릭 ⇒ repository 선택 ⇒ New 버튼 클릭
  - Repository name: 프로젝트 저장소 이름 설정 (repository명이 url 주소가 되니 작명이 중요함)
  - Description: 프로젝트에 대한 간단한 설명 입력
  - Visibility: 저장소 공개 범위 설정

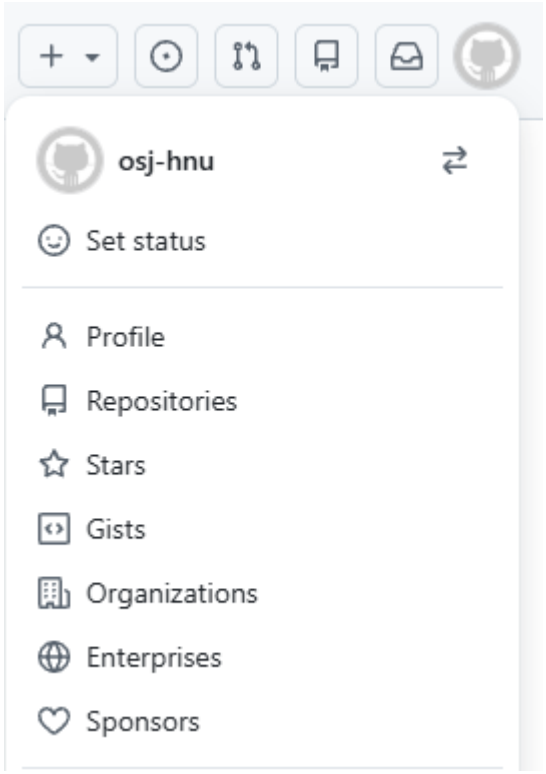
구분	Public (공개)	Private (비공개)
접근 권한	누구나 저장소 열람 가능	허용된 사용자만 접근 가능
소스코드	외부에 공개	외부 접근 제한
주요 활용	오픈소스, 개인 포트폴리오	기업 내부 프로젝트, 비공개 개발

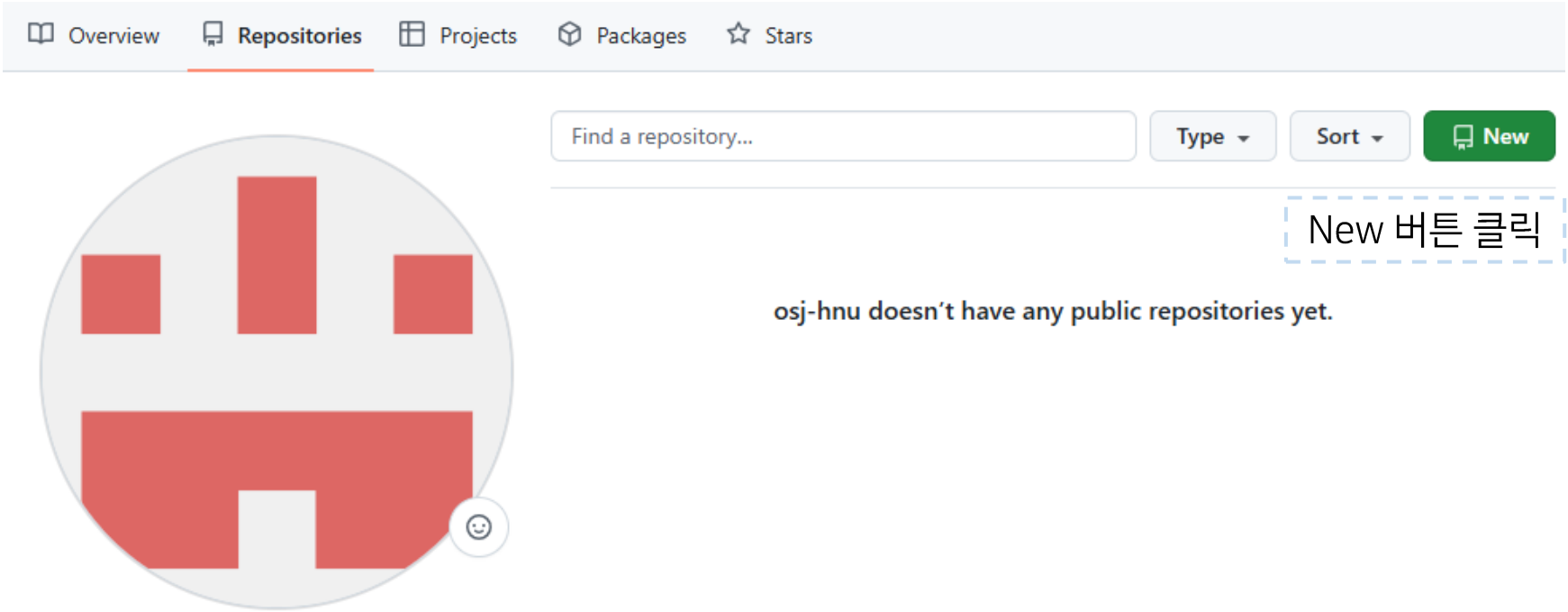
  - Create repository: 설정 완료 후 저장소 생성

02 개발 환경과 Git 활용

■ GitHub 저장소 생성 실습

- GitHub 로그인 후 프로필 클릭 ⇒ repository 선택

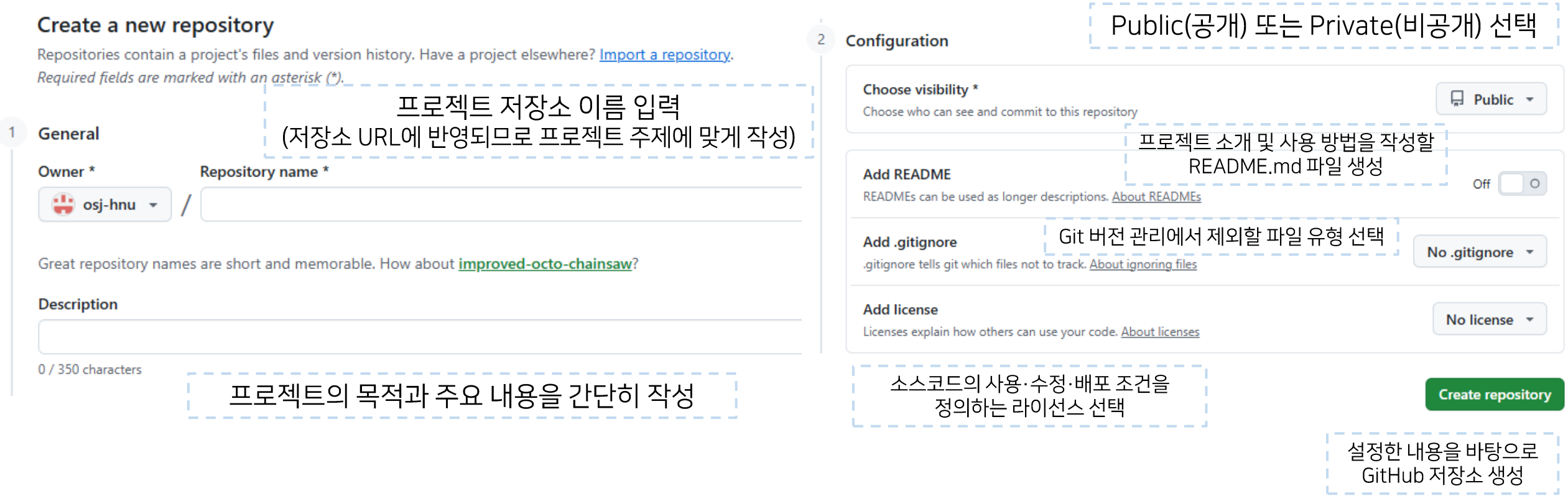




02 개발 환경과 Git 활용

■ GitHub 저장소 생성 실습

- repository 생성



02 개발 환경과 Git 활용

■ Git 기본 명령어의 이해

- Git은 명령어를 통해 저장소 생성, 변경 이력 관리 및 원격 저장소 동기화 수행
- VS Code, Eclipse 등 개발 도구에서는 주요 Git 기능을 GUI(Graphical User Interface)로 제공
- 동일한 Git 기능을 명령어 또는 개발 도구의 메뉴를 통해 실행 가능

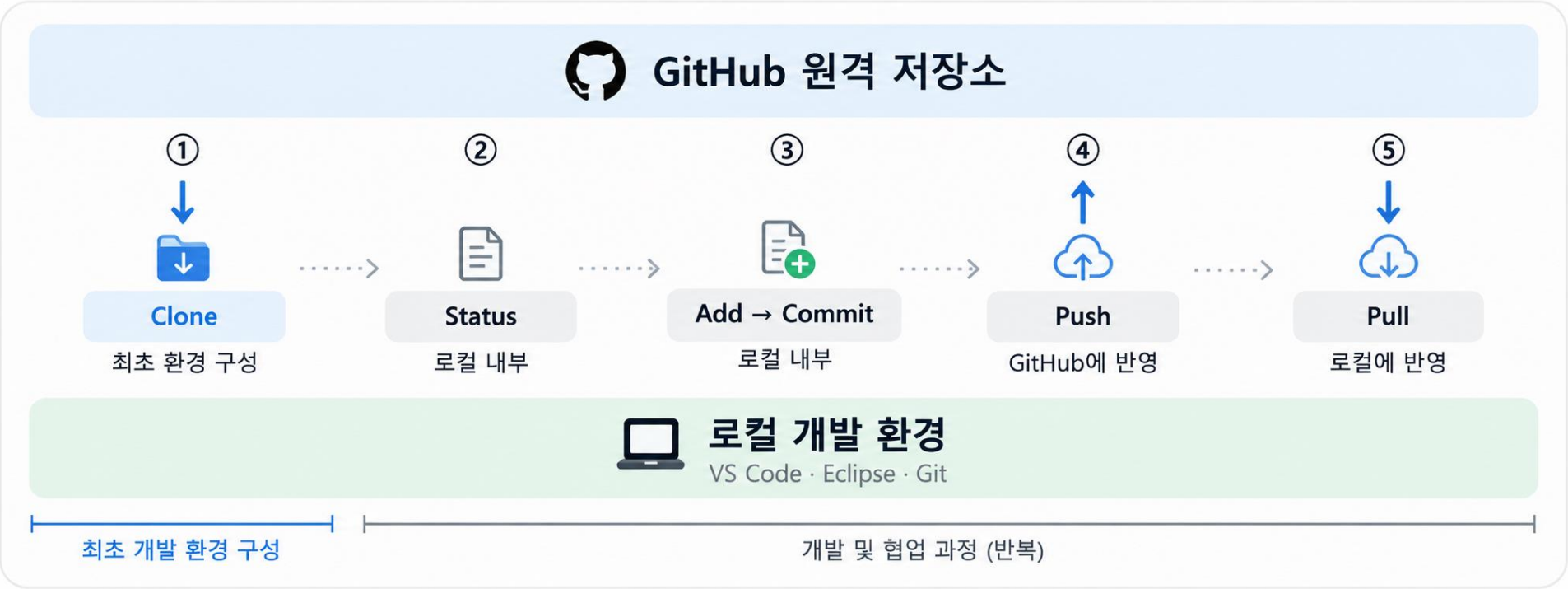
Git 명령어	주요 기능	IDE에서 제공하는 기능 예시
git init	Git 저장소 초기화	Initialize Repository
git clone	원격 저장소 복제	Clone Repository
git status	파일 변경 상태 확인	Source Control(변경 파일 및 상태 확인)
git add	변경 파일 스테이징	Stage Changes
git commit	변경 이력 기록	Commit
git push	원격 저장소에 변경 이력 반영	Push / Sync
git pull	원격 변경 사항 가져와 통합	Pull / Sync

** 메뉴 명칭과 제공 방식은 개발 도구 및 버전에 따라 다를 수 있으며, Sync는 일반적으로 Pull과 Push를 함께 수행하는 기능입니다.*

02 개발 환경과 Git 활용

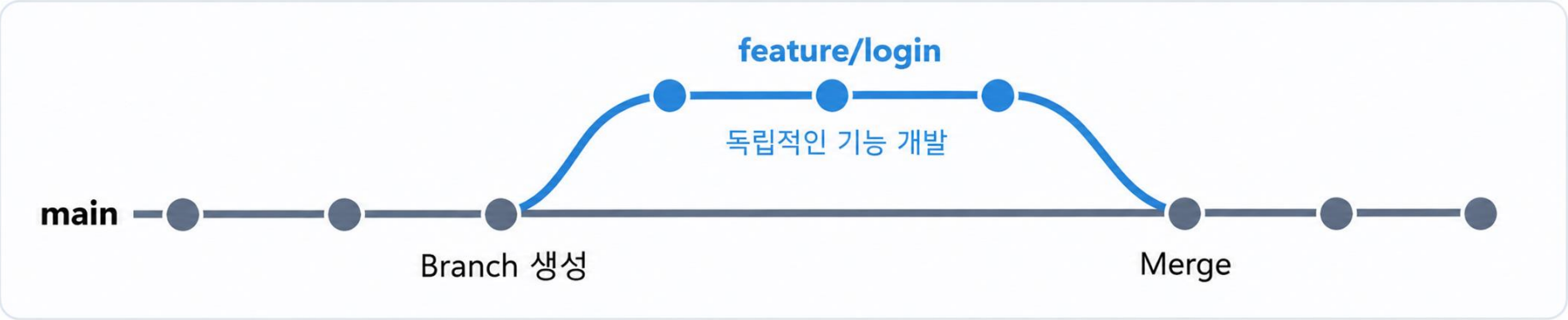
■ Git 기본 명령어의 작업 흐름

- ① 최초 개발 환경 구성: GitHub 저장소를 로컬 개발 환경으로 복제하고 원격 저장소 연결 (git clone)
- ② 파일 작성 및 변경 확인: 프로젝트 파일을 작성·수정하고 변경 상태 확인 (git status)
- ③ 변경 이력 기록: 변경 파일을 스테이징 영역에 추가하고 Commit 수행 (git add → git commit)
- ④ 원격 저장소 반영: 로컬 변경 이력을 GitHub 원격 저장소에 공유 (git push)
- ⑤ 최신 변경 사항 반영: 다른 팀원이 공유한 변경 사항을 로컬 환경에 통합 (git pull)



■ Branch와 Merge를 활용한 협업

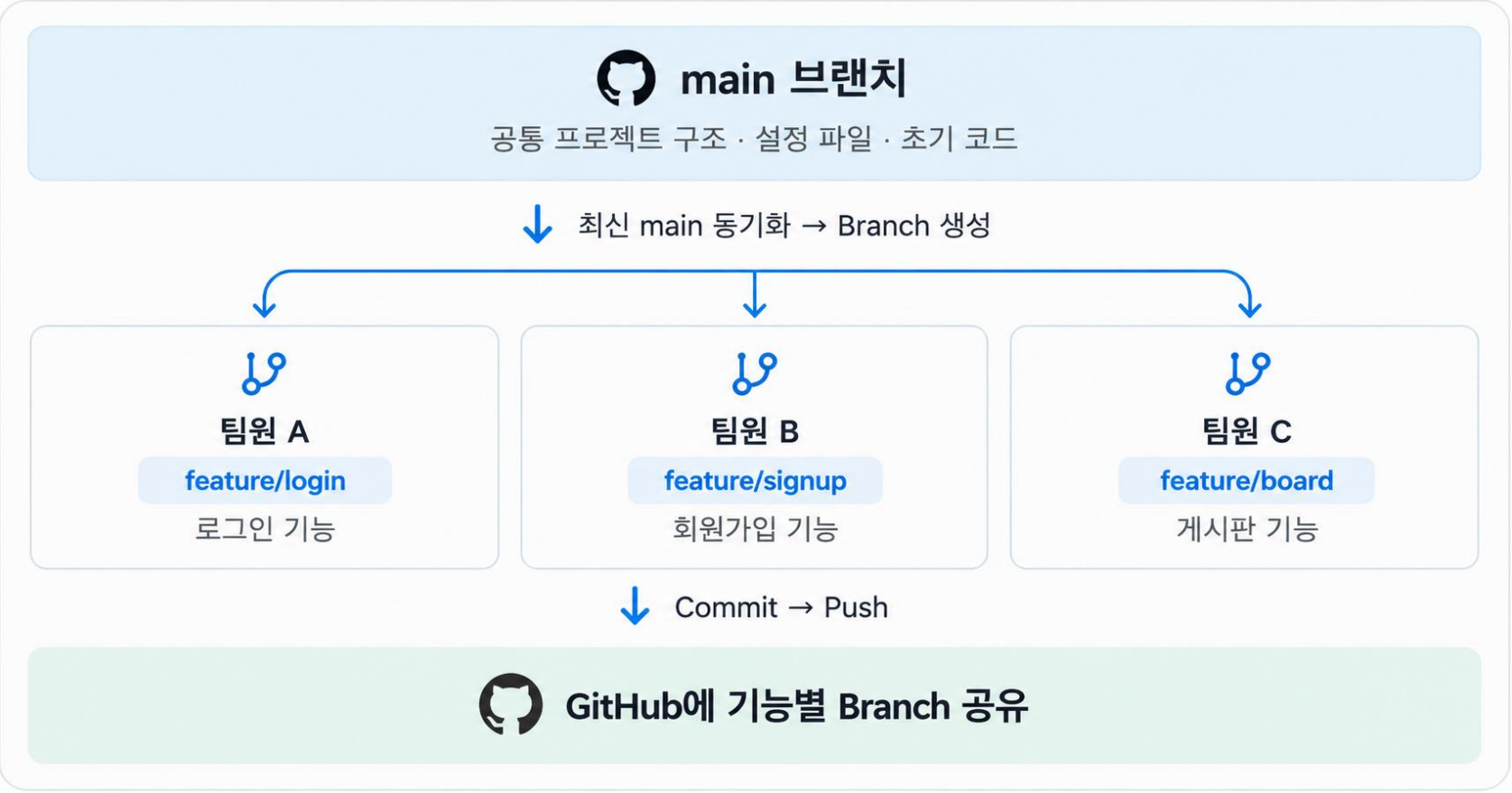
- main 브랜치와 Branch의 개념
  - main 브랜치: 프로젝트의 공통 코드와 주요 변경 이력을 관리하는 기준 브랜치
  - Branch: 기존 브랜치의 특정 Commit을 기준으로 분기하여 독립적으로 작업하는 개발 흐름
  - Branch 생성: 기존 프로젝트의 파일과 변경 이력을 기반으로 새로운 작업 브랜치 생성
  - 독립적인 개발: Branch에서 수정한 내용은 main 브랜치에 즉시 반영되지 않음
  - Merge: Branch에서 개발한 변경 사항을 main 등 다른 브랜치에 통합



** main의 기존 Commit을 기준으로 Branch를 생성하고, 개발 완료 후 변경 사항을 main에 통합*

Branch를 활용한 팀 협업

- 공통 프로젝트 구성: main 브랜치에 공통 폴더 구조, 설정 파일 및 초기 코드를 구성하고 팀원 간 공유
- 기능별 독립 개발: 최신 main을 기준으로 팀원별 Branch를 생성하고, 담당 기능 개발 후 Commit과 Push 수행



Branch는 모든 작업에서
반드시 생성해야 하는 것은 아니며,
작업의 규모와 협업 방식에 따라
선택적으로 활용할 수 있음

03 프로젝트 문서화와 GitHub 실습

■ 프로젝트 문서화의 필요성

- GitHub 저장소에서 프로젝트의 목적, 구성 및 실행 방법을 문서로 관리
- 팀원들이 동일한 프로젝트 구조와 개발 환경을 이해할 수 있도록 정보 공유
- 프로젝트에 필요한 파일과 Git으로 관리하지 않을 파일을 구분

■ 주요 문서 파일

구분	README.md	.gitignore
개념	프로젝트의 목적, 기능, 구성 및 사용 방법 등을 설명하는 Markdown 형식의 문서	Git에서 추적하거나 관리하지 않을 파일과 폴더를 지정하는 설정 파일
주요 목적	프로젝트 정보를 체계적으로 문서화하고 팀원 및 사용자에게 안내	불필요한 파일이 Git 저장소에 포함되는 것을 방지
작성 내용	프로젝트 개요, 주요 기능, 기술 스택, 폴더 구조, 설치 및 실행 방법, 팀원별 역할 등	임시 파일, 빌드 결과물, 로그 파일, 개인 개발 환경 설정 및 비밀정보가 포함된 파일 등
활용 방법	GitHub 저장소에서 프로젝트 소개 문서로 표시되며, 개발 진행에 따라 지속적으로 수정·관리	Git이 지정된 파일을 새롭게 추적하지 않도록 설정하여 저장소 관리
협업 효과	팀원들이 공통 프로젝트 구조와 개발 환경을 이해하고 동일한 기준으로 개발하도록 지원	개발 환경별 불필요한 파일 공유를 줄이고 저장소의 일관성 유지

03 프로젝트 문서화와 GitHub 실습

Markdown 기본 문법

- README.md는 Markdown 문법을 활용하여 프로젝트 정보를 작성하는 문서
- # 기호로 제목과 소제목을 구분하고, - 기호로 목록 작성
- 링크, 이미지, 코드 블록 등을 활용하여 프로젝트 관련 정보 제공
- GitHub에서는 Markdown으로 작성한 내용을 자동으로 변환하여 표시

README.md 주요 작성 항목

작성 항목	작성 내용
프로젝트 개요	프로젝트명, 개발 목적 및 해결하려는 문제
주요 기능	사용자에게 제공하는 핵심 기능과 서비스
기술 스택	프로그래밍 언어, 프레임워크, 데이터베이스 등
프로젝트 구조	주요 폴더와 파일 구성 및 역할
설치 및 실행 방법	개발 환경 설정과 프로젝트 실행 절차
팀원 및 역할	참여 인원과 담당 기능
참고 자료	화면 설계서, 관련 문서, 배포 URL 등

03 프로젝트 문서화와 GitHub 실습

■ Markdown을 활용한 README.md 작성

- README.md는 Markdown 문법으로 작성하며, GitHub에서 제목·목록·링크 등을 자동으로 변환하여 표시
- 프로젝트 개요, 주요 기능, 기술 스택, 팀원별 역할 등을 작성하여 프로젝트 정보 공유

README.md 작성 내용

```
1  # 학생 프로젝트 관리 시스템
2
3  ## 프로젝트 개요
4  팀 프로젝트의 진행 상황을
5  관리하는 웹 시스템
6
7  ## 주요 기능
8  - 로그인 및 회원가입
9  - 프로젝트 동록
10 - 진행 상황 관리
11
12 ## 기술 스택
13 - HTML, CSS, JavaScript
14 - Spring Boot, MySQL
15
16 ## 팀원 및 역할
   - 팀원 A: 로그인
   - 팀원 B: 프로젝트 관리
```

GitHub 표시 결과

학생 프로젝트 관리 시스템

프로젝트 개요

팀 프로젝트의 진행 상황을 관리하는 웹 시스템

주요 기능

- 로그인 및 회원가입
- 프로젝트 동록
- 진행 상황 관리

기술 스택

- HTML, CSS, JavaScript
- Spring Boot, MySQL

팀원 및 역할

- 팀원 A: 로그인
- 팀원 B: 프로젝트 관리

03 프로젝트 문서화와 GitHub 실습

■ .gitignore 파일 작성 및 활용

- .gitignore는 Git에서 추적하지 않을 파일과 폴더를 지정하여 불필요한 파일이 저장소에 포함되는 것을 방지
- 임시 파일, 빌드 결과물, 로그 파일 및 비밀정보가 포함된 환경 설정 파일 등을 Git 관리 대상에서 제외

.gitignore 작성 내용

파일 및 폴더 제외 규칙

gitignore

```
# 환경 변수 및 비밀정보
.env
.env.local

# 로그 파일
*.log

# 빌드 결과물
```

프로젝트 파일 관리 예시

project/

src/	관리
README.md	관리
.gitignore	관리
.env	제외
debug.log	제외
dist/	제외
.vscode/	제외

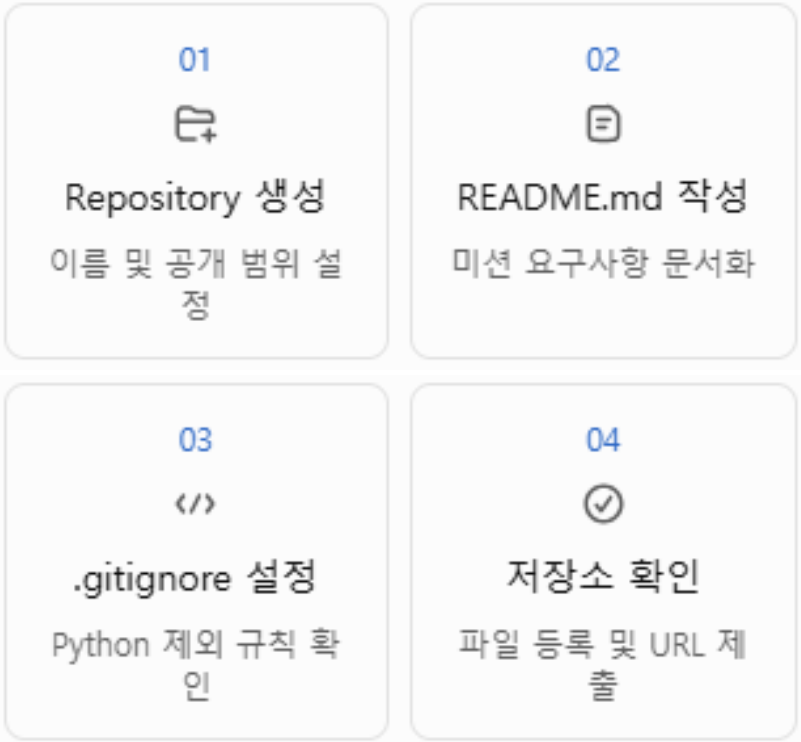
** 위 예시는 아직 Git에서 추적하지 않는 파일을 기준으로 합니다. 이미 추적 중인 파일은 .gitignore에 추가해도 자동으로 제외되지 않습니다.*

03 프로젝트 문서화와 GitHub 실습

GitHub 프로젝트 저장소 구성 실습 목표

- Python 퀴즈 게임 개발을 위한 GitHub Repository 생성
- README.md와 .gitignore 작성 및 저장소 등록
- 팀원들이 동일한 프로젝트 구조와 개발 기준을 공유할 수 있도록 초기 환경 구성

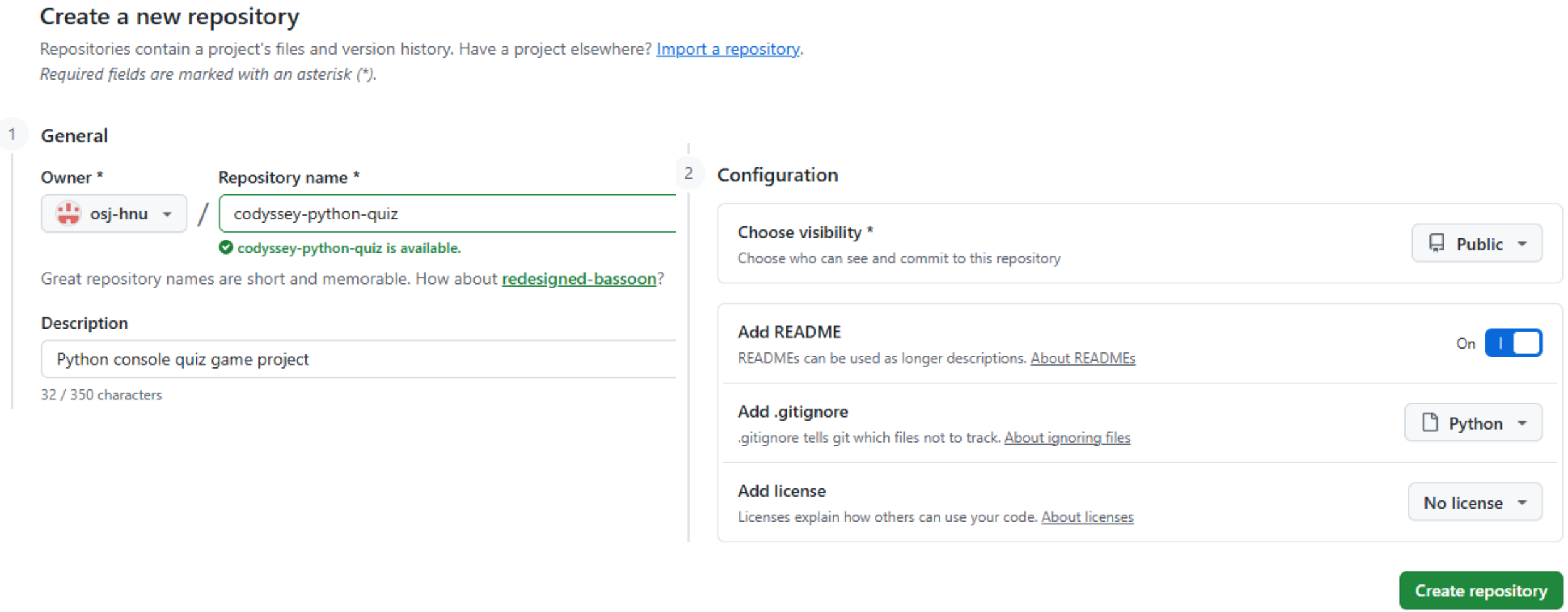
항목	실습 내용
프로젝트명	Python 콘솔 퀴즈 게임
Repository 이름	codyssey-python-quiz
Description	Python console quiz game project
공개 범위	Public
README.md	생성 시 추가
.gitignore	Python 템플릿 선택
License	선택하지 않음



03 프로젝트 문서화와 GitHub 실습

■ GitHub Repository 생성

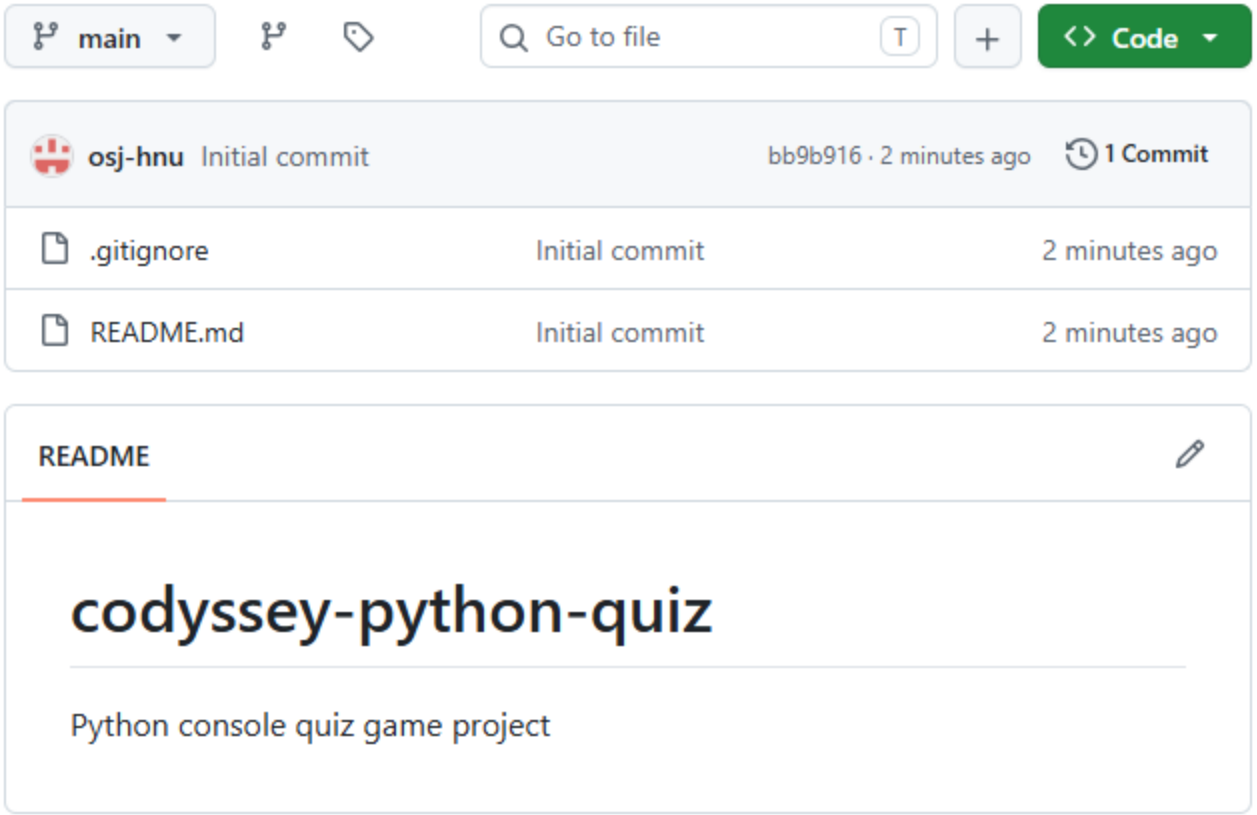
- Python 퀴즈 게임 개발을 위한 GitHub Repository codyssey-python-quiz 생성

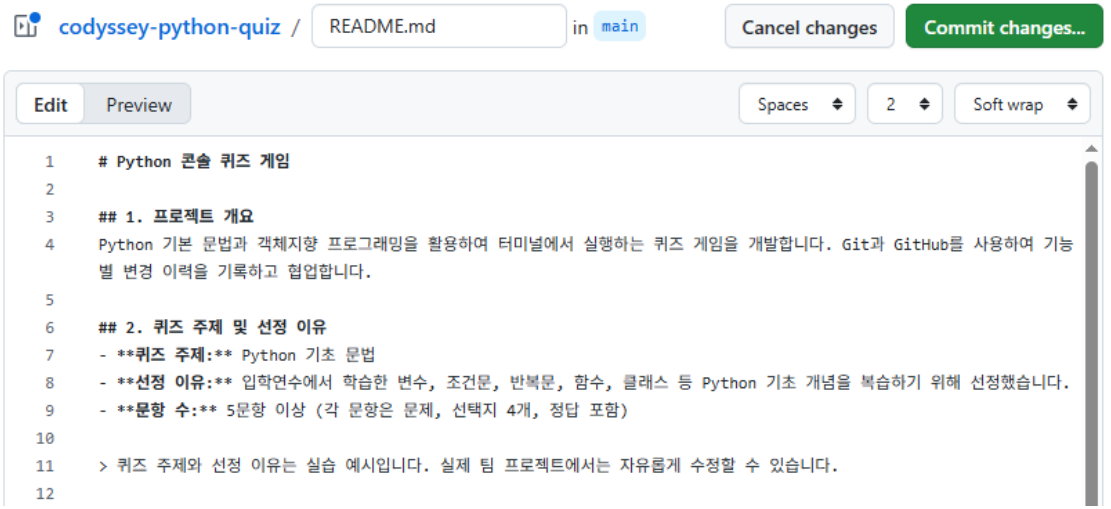


03 프로젝트 문서화와 GitHub 실습

■ README.md 초기 생성 확인

- 생성된 codyssey-python-quiz 레파지토리에 README.md 파일 생성 확인

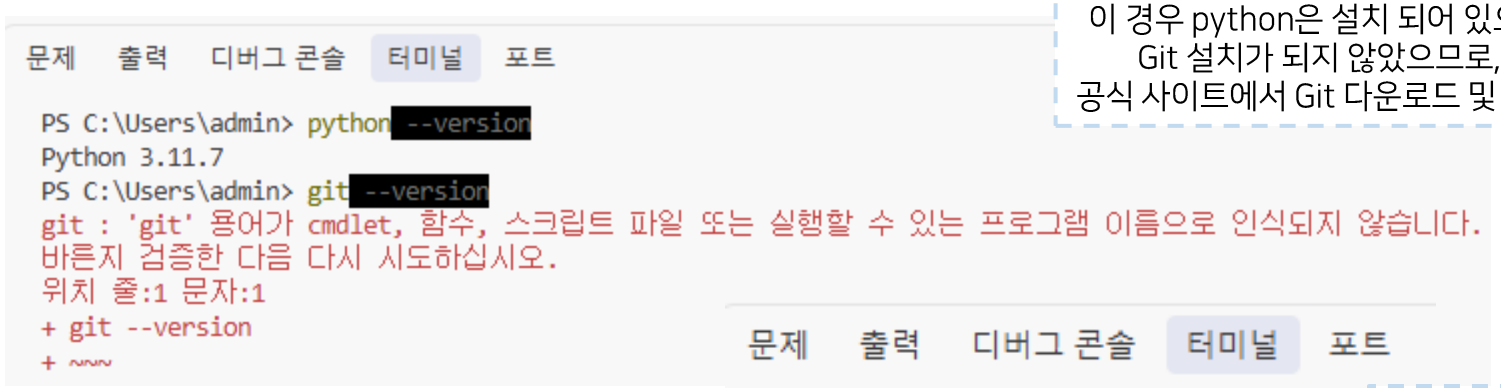




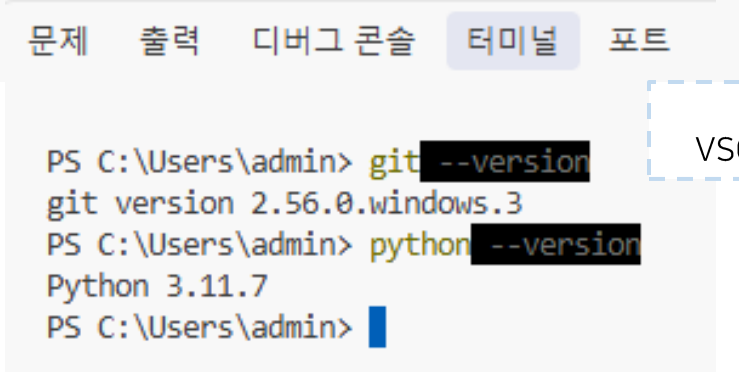
03 프로젝트 문서화와 GitHub 실습

■ VSCode 환경 설정 및 GitHub Repository 복제

- ① Python, Git 설치 및 실행 환경 확인
- VS Code 실행 후 상단 메뉴에서 [Terminal] → [New Terminal]을 선택하여 터미널 열기(Ctrl + Shift + `)
- python --version 명령어로 Python 설치 여부 확인 후 설치되지 않았다면 python 설치 (https://www.python.org/downloads/)
- git –version 명령어 실행으로 Git 설치 여부 확인 (https://git-scm.com/) 설치 후 VSCode 재실행



이 경우 python은 설치 되어 있으나
Git 설치가 되지 않았으므로,
공식 사이트에서 Git 다운로드 및 설치

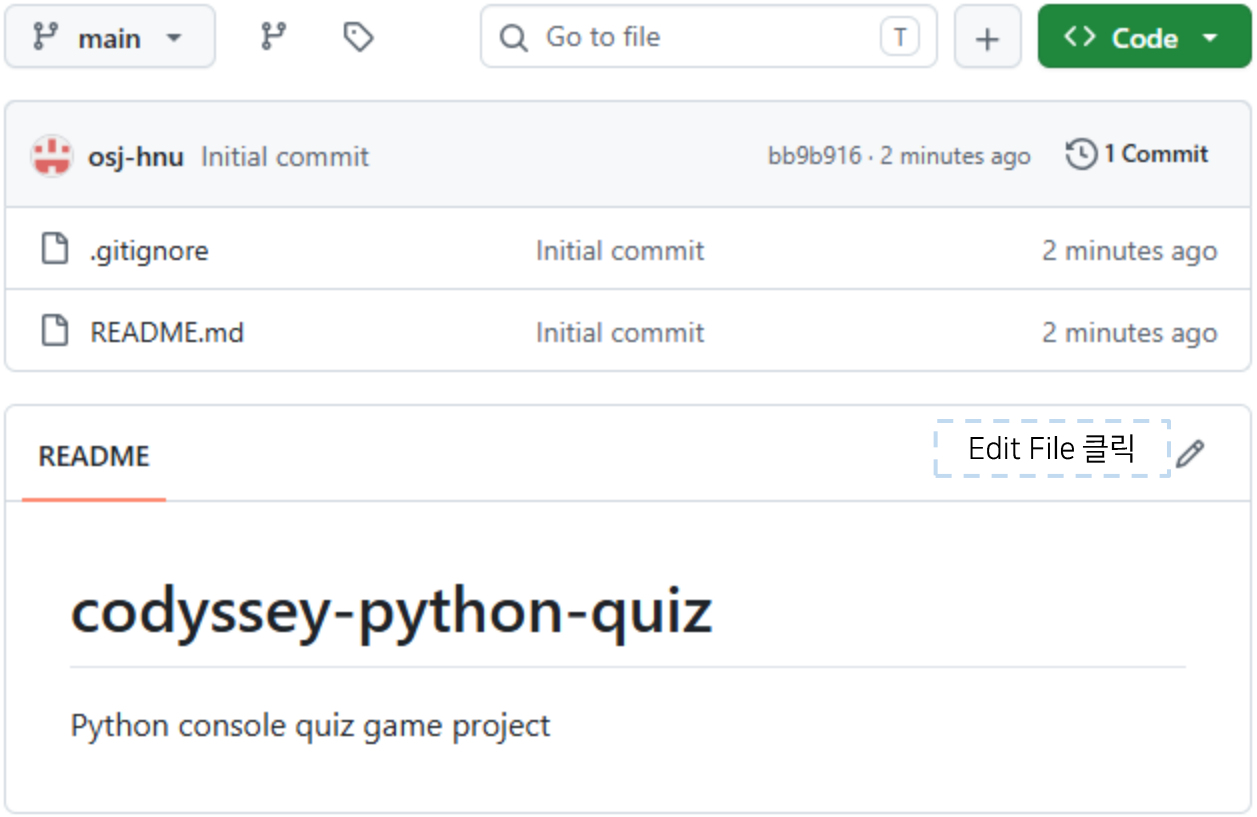


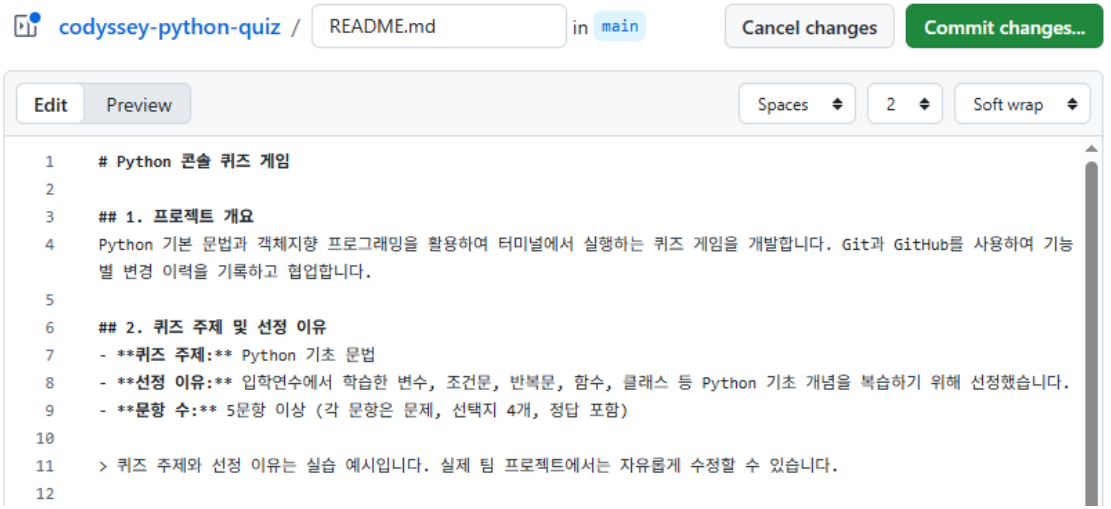
python과 Git 설치 후
VSCode 재실행하여 터미널에 명령어 다시 실행

03 프로젝트 문서화와 GitHub 실습

■ README.md 작성 실습

- 생선된 codyssey-python-quiz 레파지토리에 README.md 파일 수정

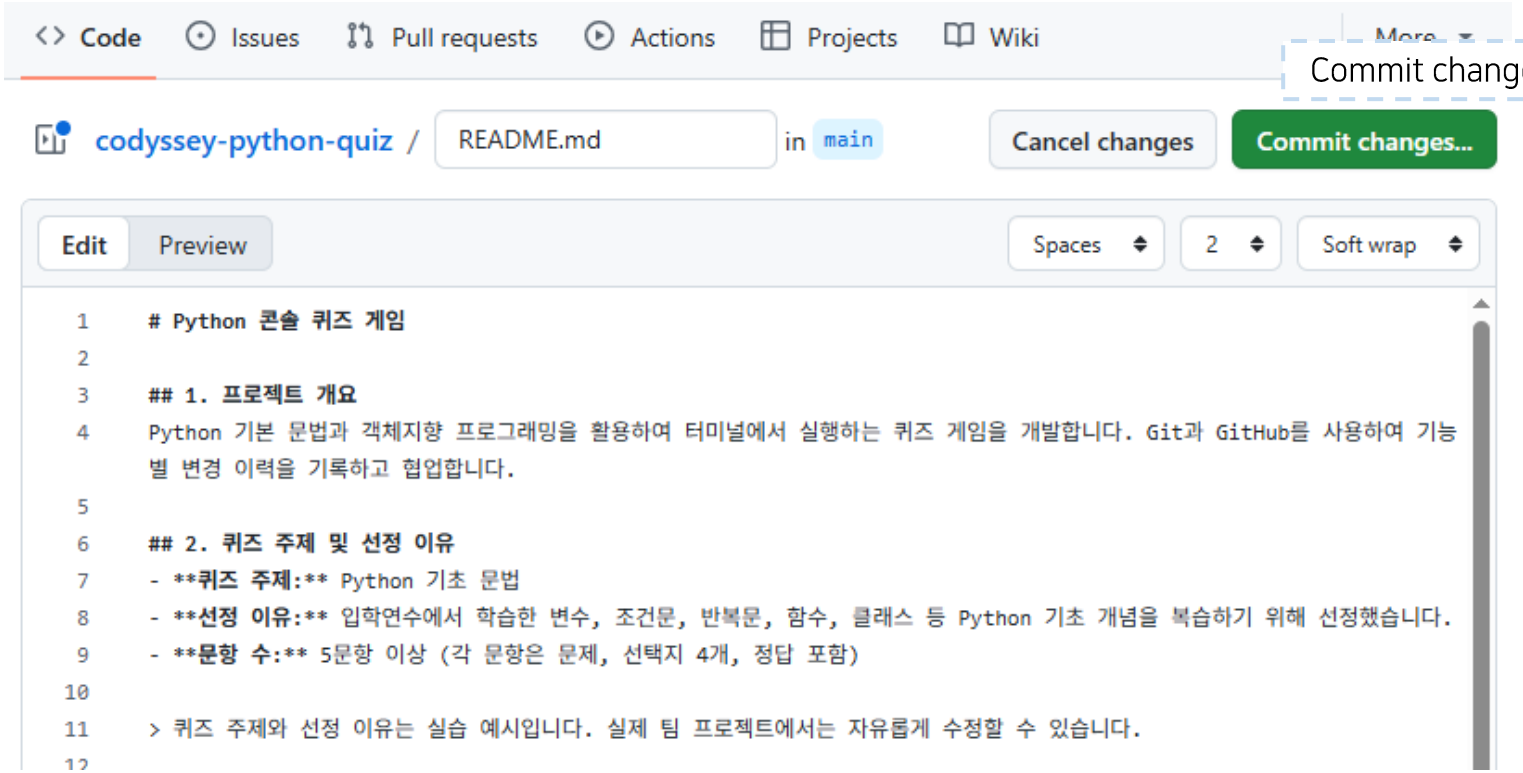


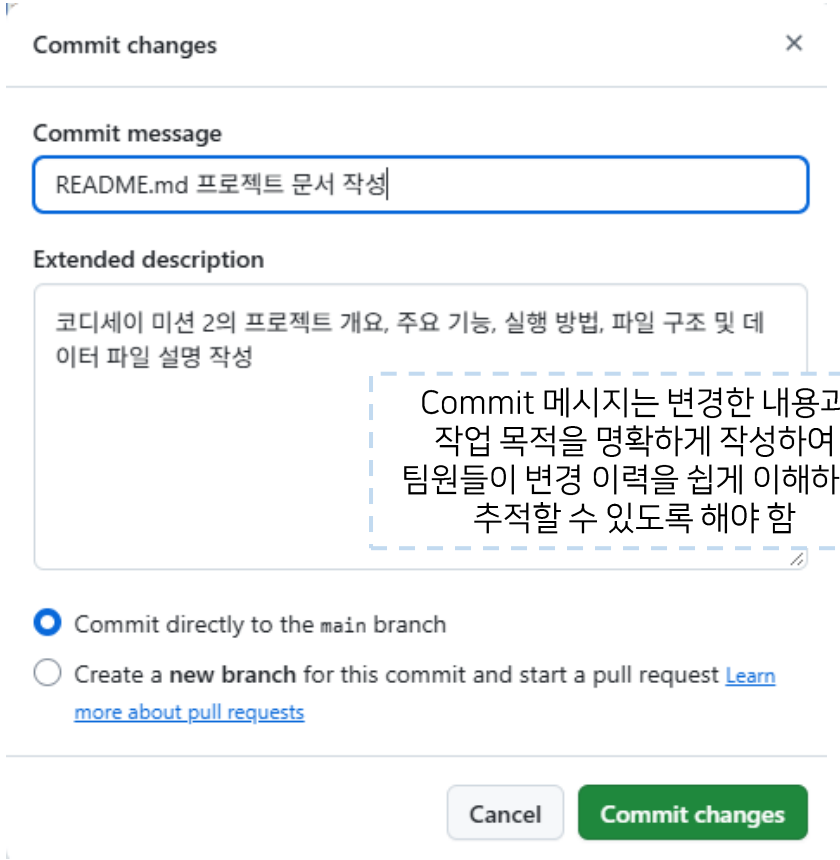


03 프로젝트 문서화와 GitHub 실습

■ README.md 작성 실습

- 생선된 codyssey-python-quiz 레파지토리에 README.md 파일 수정하고, Commit 메시지 작성하기





03 프로젝트 문서화와 GitHub 실습

■ README.md 작성 실습

- 작성된 README.md 확인하기

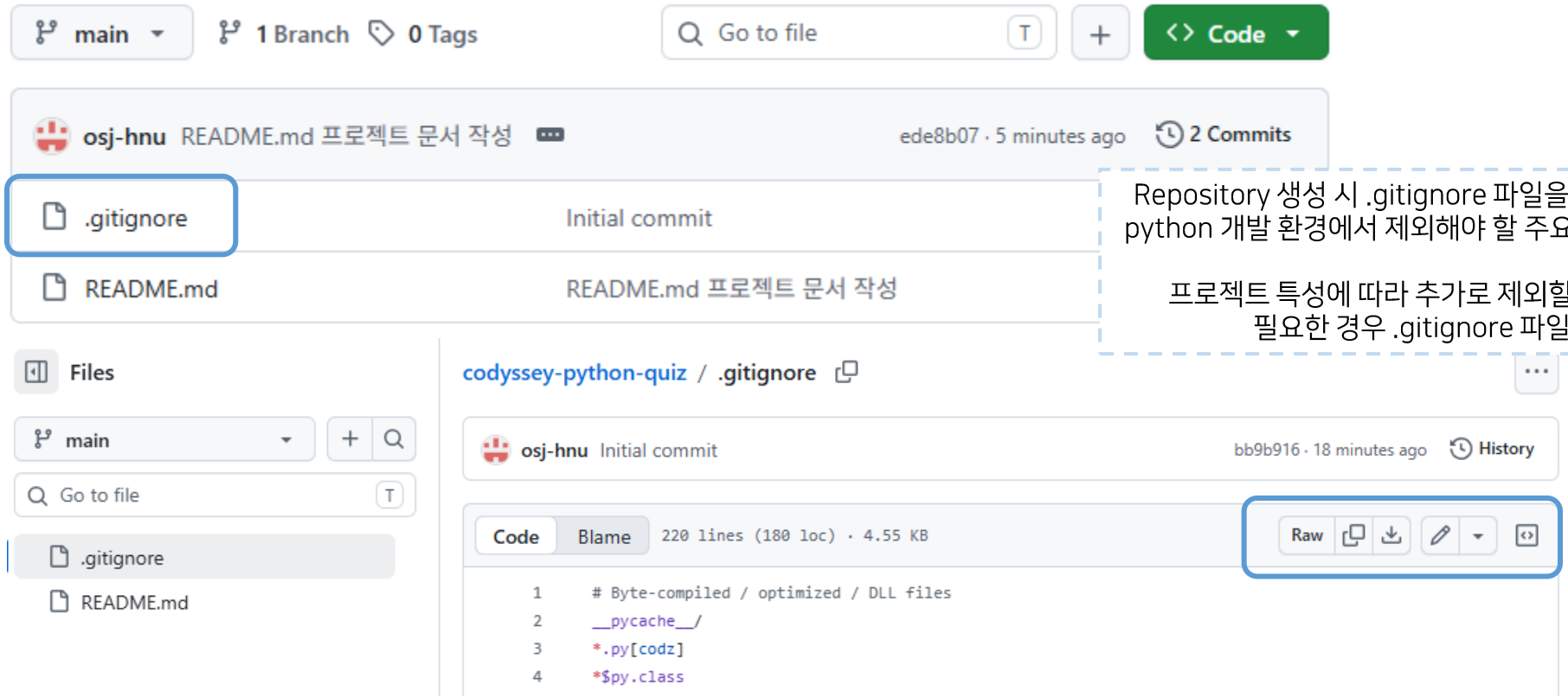
https://github.com/osj-hnu/codyssey-python-quiz



03 프로젝트 문서화와 GitHub 실습

GitHub에서 .gitignore 파일 작성

- main 브랜치에서 .gitignore 파일 클릭
- 연필 모양의 Edit 버튼을 클릭하여 Python 개발 환경에 맞는 제외 규칙 작성
- Commit changes 버튼을 클릭하여 main 브랜치에 반영



Repository 생성 시 .gitignore 파일을 Python 템플릿으로 선택하면, python 개발 환경에서 제외해야 할 주요 파일과 폴더가 자동으로 작성됨

프로젝트 특성에 따라 추가로 제외할 파일이나 폴더를 확인하고, 필요한 경우 .gitignore 파일을 수정하여 적용함.

Thank you for your attention